Міністерство освіти і науки України

Національний технічний університет України

Київський політехнічний інститут імені Ігоря Сікорського

Лабораторна робота №2

« Перетворення ER-діаграми на схему PostgreSQL »

Роботу виконали

Студент групи ІО-45

Соловійщук

Назар

Роботу перевірив:

Русінов В.В.

Київ 2026

ЛАБОРАТОРНА РОБОТА № 2

Тема: Перетворення ER-діаграми на схему PostgreSQL

Мета: Навчитися перетворювати концептуальну ER-модель у реляційну схему бази даних, створювати таблиці в PostgreSQL з використанням SQL DDL-інструкцій, визначати типи даних, ключі та обмеження, а також наповнювати базу тестовими даними.

Опис схеми бази даних

База даних розроблена для системи прокату фільмів і складається з 5 основних таблиць:

movies: містить інформацію про назви, жанри та тривалість фільмів. Поле duration має обмеження CHECK > 0.

inventory: облік фізичних копій дисків, прив'язаних до конкретних фільмів через зовнішній ключ movie_id.

customers: дані клієнтів. Поле email має обмеження UNIQUE для запобігання дублікатів.

rentals: веде облік видачі та повернення дисків. Містить зовнішні ключі на клієнта та інвентар для забезпечення цілісності даних.

payments: фінансові операції. Поле amount має обмеження CHECK (amount >= 0)

SQL-скрипт створення та наповнення

```
-- Створення таблиць (DDL)
CREATE TABLE movies (
    movie_id SERIAL PRIMARY KEY,
    title VARCHAR(255) NOT NULL,
    release_year INTEGER,
    genre VARCHAR(100),
    director VARCHAR(255),
    duration INTEGER CHECK (duration > 0)
);

CREATE TABLE inventory (
    inventory_id SERIAL PRIMARY KEY,
    movie_id INTEGER NOT NULL REFERENCES movies(movie_id),
    store_id INTEGER NOT NULL,
    condition VARCHAR(50)
);

CREATE TABLE customers (
    customer_id SERIAL PRIMARY KEY,
    first_name VARCHAR(100) NOT NULL,
    last_name VARCHAR(100) NOT NULL,
    phone VARCHAR(20),
    email VARCHAR(255) UNIQUE NOT NULL,
    address TEXT,
    registration_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE rentals (
    rental_id SERIAL PRIMARY KEY,
```

```sql
    customer_id INTEGER NOT NULL REFERENCES customers(customer_id),
    inventory_id INTEGER NOT NULL REFERENCES inventory(inventory_id),
    rental_date TIMESTAMP NOT NULL,
    return_due_date TIMESTAMP NOT NULL,
    return_date TIMESTAMP
);

CREATE TABLE payments (
    payment_id SERIAL PRIMARY KEY,
    rental_id INTEGER NOT NULL REFERENCES rentals(rental_id),
    payment_type VARCHAR(50) NOT NULL,
    amount NUMERIC(10, 2) NOT NULL CHECK (amount >= 0),
    payment_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Наповнення даними (INSERT)
INSERT INTO movies (title, release_year, genre, director, duration) VALUES
('Inception', 2010, 'Sci-Fi', 'Christopher Nolan', 148),
('The Godfather', 1972, 'Crime', 'Francis Ford Coppola', 175),
('Pulp Fiction', 1994, 'Crime', 'Quentin Tarantino', 154),
('The Matrix', 1999, 'Sci-Fi', 'Lana Wachowski', 136),
('Interstellar', 2014, 'Sci-Fi', 'Christopher Nolan', 169);

INSERT INTO inventory (movie_id, store_id, condition) VALUES
(1, 1, 'New'), (1, 1, 'Good'), (2, 1, 'New'), (3, 2, 'Fair'), (4, 2, 'Good');

INSERT INTO customers (first_name, last_name, phone, email, address) VALUES
('Nazar', 'Solovishchuk', '380991234567', 'nazar@example.com', 'Kyiv, Ukraine'),
('Alice', 'Smith', '380997654321', 'alice@example.com', 'London, UK'),
('Bob', 'Johnson', '380501112233', 'bob@example.com', 'New York, USA');

INSERT INTO rentals (customer_id, inventory_id, rental_date, return_due_date,
return_date) VALUES
(1, 1, '2026-03-01 10:00:00', '2026-03-08 10:00:00', '2026-03-07 12:00:00'),
(2, 3, '2026-03-05 14:30:00', '2026-03-12 14:30:00', NULL),
(3, 4, '2026-03-10 09:00:00', '2026-03-17 09:00:00', NULL);

INSERT INTO payments (rental_id, payment_type, amount, payment_date) VALUES
(1, 'Rental Fee', 50.00, '2026-03-01 10:05:00'),
(1, 'Late Fee', 5.00, '2026-03-07 12:05:00'),
(2, 'Rental Fee', 45.00, '2026-03-05 14:35:00'),
(3, 'Rental Fee', 60.00, '2026-03-10 09:10:00');
```

Скріншот №1: Список створених таблиць у базі даних `movie_rental`

Скріншот №2: Перевірка вмісту таблиці `rentals`

Query Query History

```
1 SELECT * FROM rentals;
```

Data Output Messages Notifications

Showing rows: 1 to

	rental_id [PK] integer	customer_id integer	inventory_id integer	rental_date timestamp without time zone	return_due_date timestamp without time zone	return_date timestamp without time zone
1	1	1	1	2026-03-01 10:00:00	2026-03-08 10:00:00	2026-03-07 12:00:00
2	2	2	3	2026-03-05 14:30:00	2026-03-12 14:30:00	[null]
3	3	3	4	2026-03-10 09:00:00	2026-03-17 09:00:00	[null]

Рисунок 3 — Підтвердження успішного виконання SQL-скрипта.

Data Output Messages Notifications

```
Successfully run. Total query runtime: 145 msec.
3 rows affected.
```

Висновок:
У ході виконання лабораторної роботи було успішно реалізовано реляційну модель бази даних на основі раніше розробленої ER-діаграми. Були застосовані обмеження цілісності (PRIMARY KEY, FOREIGN KEY, NOT NULL, CHECK, UNIQUE). База даних була протестована шляхом вставки та вибірки даних, що підтвердило коректність зв’язків між сутностями.